Vision is the bottleneck: seven action-chunking tricks for a VLA, re-measured on a Jetson Orin NX

Ravindu Somawansa
Shisso Robotics
ravindu.somawansa@shissorobotics.com

Abstract. We re-measured seven training-free methods for action chunking in flow-matching VLAs, among them exact real-time chunking, speculative chunk selection and entropy-adaptive chunk length. Their papers report numbers from desktop GPUs. We ran them where a low-cost robot runs its policy, with SmolVLA on a Jetson Orin NX. All tests share one checkpoint and the same recorded episodes. Each test had its threshold committed before the run, and every number traces to a result file and a protocol commit. On this board the vision encoder takes 151.0 of 219.0 ms of server time per chunk (25 W). The two methods that act on the seam between chunks pass. A gradient-free prefix overwrite cuts the boundary jump by 72 % or more. Movement-primitive seam conditioning removes most of the seam jerk. The sampling methods do not transfer on vla.cpp, which samples in sequence, so every N-sample method pays for N chunks one after another. That is the engine and not the board. In PyTorch on the same Orin, four candidates that share one vision and prefill pass and one batched denoising loop cost 1.03 chunks.

Code and rows: `github.com/ravediamond/paper-vla-edge-inference` **Reproduce:** `make check` **Protocols:** `repro/*/PROTOCOL.md`. Every number carries its source file and protocol commit.

1 Introduction

Several training-free chunking tricks for flow-matching VLAs came out this year, and their numbers come from desktop or data-centre GPUs (RTX 4090, A100, H100, RTX 5090). On these a chunk costs barely tens of milliseconds and spare compute is plentiful. A robot people can buy runs its policy on an edge GPU. There most of the chunk goes to the vision encoder, which these tricks don't affect. We measured how much of each trick is left on such a board.

Our robot is an SO-101 arm [3, 14] with a Jetson Orin NX 16 GB [2]. It runs SmolVLA [10] through vla.cpp [4], a C++ inference engine, on episodes recorded with LeRobot [1]. This paper started once SmolVLA ran on the arm through vla.cpp. We then looked at where the rest of a chunk's time went, and at which published tricks could still help. We took seven of them. Each one is a gate with a bar committed before the number. Offline effects get an episode bootstrap, and latencies are medians of at least 50 calls. The appendix maps every number to its result file and protocol commit.

2 Where the time goes

We follow one chunk of ten Euler steps at full power (MAXN_SUPER). It costs 237.5 ms end to end (Fig. 1). Vision takes 150.7 ms and prefill 14.9 ms. Each denoise step takes 5.3 ms. At 25 W the server-side split is 151.0 ms of vision in 219.0 ms.

Finding. A trick that only touches the denoising loop works on 22 % of a chunk at most.

The Orin ran in two power modes for this paper. The serving split and the boundary experiments ran at 25 W. The latency experiments ran at full power. The protocol of each gate froze its mode, and every number below carries it.

3 Seven tricks, one checkpoint

All offline effects use the same SmolVLA checkpoint and the same 20 recorded episodes, with 254 chunk boundaries per delay. Table 1 gives every effect, bar and verdict.

Two tricks act on the seam between chunks, and both pass. We start with the prefix overwrite, which replaces the executed prefix of the new chunk with the actions already committed. We ran it in PyTorch and in vla.cpp and it passes the 50 % jump-cut bar at every delay. In vla.cpp it adds 0.4 ms per chunk. The second is movement-primitive chunking [8]. We projected each chunk onto ten Gaussian bases, which keeps the error within its bar and removes within-chunk jerk. Conditioning on the last action sent removes 83–87 % of the seam jerk at +0.19° of error, and the paper reports 73–84 %. We measured on the commanded stream only.

The overwrite needs no gradient. In our PyTorch reference (`repro/chunk-boundary/scripts/run_g13b_rtc_offline.py`) it is the last branch of the guided denoising step, where `W` holds the prefix weights and `Y` the committed actions:

```
        with torch.no_grad():
            v = original_denoise_step_partial(x)
        x1 = x - t * v
        err = (Y - x1) * W
        if self.method == "identity":
            out = v - guidance_weight(t, self.beta) * err
        else:  # overwrite: x1_hat <- x1_hat + W (Y - x1_hat),
    v' = (x_t - x1_hat') / t
            out = v - err / max(t, 1e-6)
```

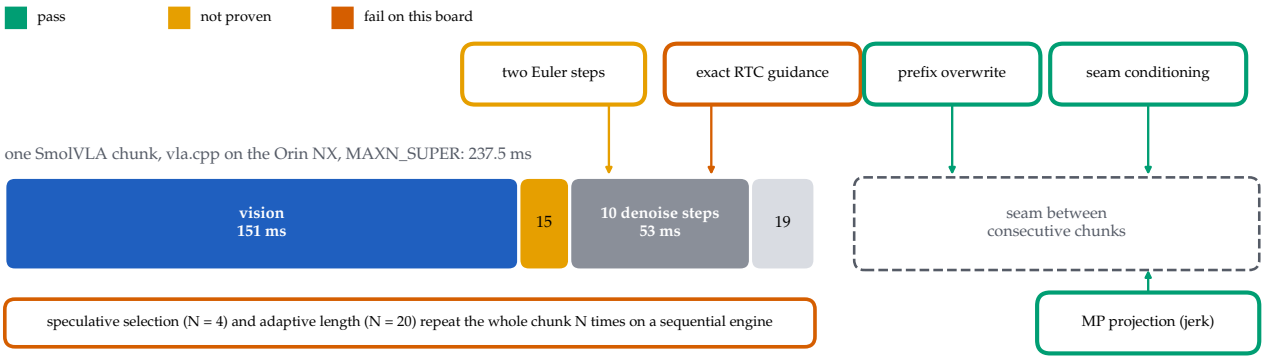


Figure 1: **Where one SmolVLA chunk's time goes on the Orin NX and where each trick acts.** One chunk takes 237.5 ms with vla.cpp at MAXN_SUPER: vision 150.7 ms, prefill 14.9 ms, ten denoise steps 53 ms, and a rest. Four tricks act on the seam between consecutive chunks or on the chunk's smoothness and pass their bars (green). Two act inside the denoising loop: cutting ten Euler steps to two is not proven, and exact guidance fails its bar. Speculative selection and adaptive length sample the chunk N times, which costs N chunks in series on vla.cpp (red).

Exact real-time chunking [5] inpaints the prefix with guidance instead of overwriting it. We ran it and it cuts the jump by less than 50 % at every delay (Fig. 2). In PyTorch it costs 1.9× a naive chunk.

We ran the reduction of Euler steps from ten to two [12] and it saves 41 ms per chunk at full power. The vision cost is much larger. The paper claims that the action cloud contracts around an unchanged centroid, with a cloud ratio of 0.51 on π_0. Our ratio is 0.78. That is between the bar of 0.70 and the fail line of 0.85, so the claim is not proven here. The paper also expects fewer steps to remove edge-GPU thermal throttling. Our Orin did not throttle at ten steps in either power mode. We keep ten steps, as the protocol says while the contraction is unproven.

Two tricks sample several chunks per decision. Speculative chunk selection [16] samples four candidates. It uses the pre-commit frame to pick the one closest to the recorded teleoperation continuation. Offline it beats a random pick (one continuation per frame, training-set episodes, open loop). Its paper promises no added control-step delay. On the Orin, vla.cpp samples the candidates in sequence. A cycle costs 1,200 ms and 37 control ticks, against 8 for a naive chunk (Fig. 3). Entropy-adaptive chunk length [7] samples 20 chunks per decision. One decision takes 4,751 ms on the Orin. The threshold was 298 ms. The chosen length also sits at the floor in both task phases. The jump is no better than with fixed chunks and the overwrite. We then took the sampling out of the engine. In PyTorch on the Orin at MAXN_SUPER, one chunk takes 1,060 ms, and four chunks in series take 4,239 ms. Four candidates that share one vision and prefill pass and one batched denoising loop take 1,092 ms, which is 1.03 times one chunk. The bar was 2.0, so the series cost of N samples belongs to a sequential engine and not to this GPU. Twenty candidates take 1,803 ms batched, 1.70 times one chunk, against 21,212 ms in series. The extra GPU memory is 49 MB for four candidates and 166 MB for twenty. A PyTorch chunk costs 4.5 times a vla.cpp chunk, so these rows are not mixed with the vla.cpp rows.

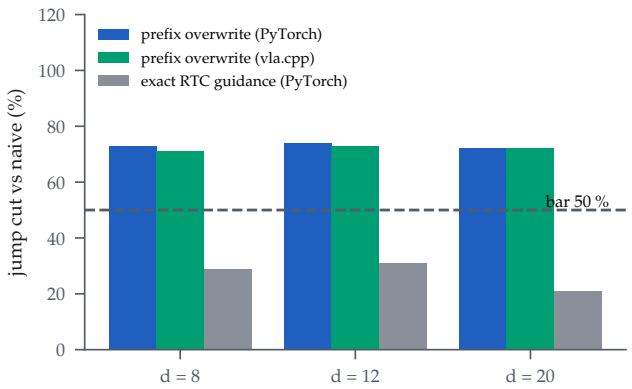


Figure 2: Median boundary-jump cut against naive asynchronous execution (254 boundaries per delay, 25 W): the overwrite clears the 50 % bar at every delay, exact RTC guidance does not.

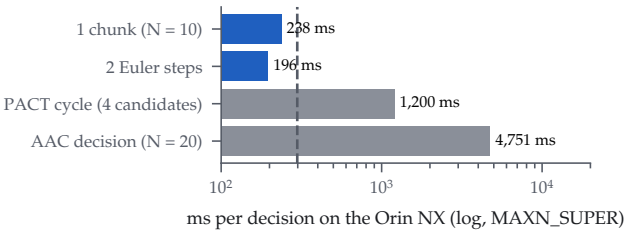


Figure 3: Cost per decision on the Orin NX (log scale, MAXN_SUPER): both sampling methods miss their bar (dashed: AAC's 298 ms; PACT's bar is one chunk, 237.5 ms).

4 What transfers to the board

What transfers. The prefix overwrite and the primitive projection with seam conditioning act on the seam. Each costs under a millisecond on this board. We keep ten Euler steps until the contraction passes its threshold. The two-step saving is small next to vision. Exact guidance, speculative selection and adaptive length all pay for N chunks in series on a sequential engine. A batched sampler does not, since four candidates cost 1.03 chunks in PyTorch on this board. If you run one of them on an edge board, check first whether your engine samples in a batch, because vla.cpp does not. This version is offline and latency only. The closed-loop columns on the arm, for seam smoothing and for the overwrite in the robot client, belong to a longer version.

5 Related work

Other chunking tricks are also reported on desktop-class GPUs. We did not re-measure them here. One extends the executing chunk instead of resampling [13]. One drives reactive chunking with a latent world model [6]. One fine-tunes the policy for latency with reinforcement learning [15]. On the engine side, TurboVLA [11] pushes VLA throughput on a desktop GPU. vla.cpp [4], the engine we measure, runs VLAs on edge boards. Two works use the same arm to evaluate policies. A standardized benchmark reports failures and recoveries of VLA policies [14]. A fleet of SO-101 cells evaluates policies at scale [9]. We re-measure the execution tricks around one policy, under bars fixed before the run.

6 Limitations

The sampling result depends on the engine. vla.cpp samples sequentially. We contribute to vla.cpp, and the overwrite runs on our fork (`ravediamond/vla.cpp`). Parity and determinism checks against PyTorch on the same board are in the gate logs. The batched control ran in PyTorch because vla.cpp has no batched sampler, so the vla.cpp rows keep their sequential costs. It used five observations of the bowls task. Effects other than latency are offline. They come from 20 recorded demonstrations of one task family (bowls), and the policy was trained on them. The arm gates for seam smoothing and for the overwrite in the robot client have not run. We left out the adaptive-length arm gate because SmolVLA is at ceiling on this task, so a gain could not show. Latency rows run at MAXN_SUPER, and the serving and boundary rows at 25 W. Power mode is a confound wherever the two are compared. The study covers one policy, SmolVLA at 450M. π_0-class models do not fit on the Orin.

References

[1] LeRobot. https://github.com/huggingface/lerobot.

[2] NVIDIA Jetson Orin NX 16 GB. https://developer.nvidia.com/embedded/jetson-orin.

[3] SO-ARM100 (SO-101 low-cost robot arm). https://github.com/TheRobotStudio/SO-ARM100.

[4] vla.cpp. https://github.com/VinRobotics/vla.cpp. C++ inference engine for VLA policies; the engine commit of each measurement is in the gate logs.

[5] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-Time Execution of Action Chunking Flow Policies, 2025. arXiv:2506.07339.

[6] Wenxi Chen, Kaidi Zhang, Chi Lin, Zhiyuan Zhang, Yu She, Yuejiang Liu, Raymond A. Yeh, Shaoshuai Mou, and Yan Gu. DREAM-Chunk: Reactive Action Chunking with Latent World Model, 2026. arXiv:2606.18589.

[7] Yuanchang Liang, Xiaobo Wang, Kai Wang, Shuo Wang, Xiaojiang Peng, Haoyu Chen, David Kim Huat Chua, and Prahlad Vadakkepat. Adaptive Action Chunking at Inference-time for Vision-Language-Action Models, 2026. arXiv:2604.04161; CVPR 2026.

[8] Shota Nomura, Takayuki Osa, Yasuhiro Kato, Kazuki Ota, Naoki Shitanda, and Tatsuya Harada. Smooth Action Chunking Via Movement-Primitive Representations for Imitation Learning, 2026. arXiv:2610.39451.

[9] Praveen Selvaraj, Lorenzo Uttini, and Ville Kuosmanen. ArmnetBench v0.1: Parallel Real-World Evaluation of Manipulation Policies on a Low-Cost Arm Farm, 2026. arXiv:2607.24481.

[10] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA:

A Vision-Language-Action Model for Affordable and Efficient Robotics, 2025. arXiv:2506.01844.

[11] Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, and Han Ding. TurboVLA: Real-Time Vision-Language-Action Model at 32 Hz on an RTX 4090 with <1 GB VRAM, 2026. arXiv:2607.27205.

[12] Jiabing Yang, Yuan Xu, Yixiang Chen, Qisen Ma, Peiyan Li, Kai Wang, Chenhang Cui, Tao Yu, Zichen Wen, Wen Ye, Bowen Fang, Ziheng He, Xiangnan Wu, Yingda Li, Yan Huang, and Liang Wang. Coupling Execution Horizon and Sampling Depth in Flow-Matching VLA Policies, 2026. arXiv:2610.32102.

[13] Seonghoon Yu, Dongwon Kim, HyungRok Jung, Yoonjae Baek, Byung kwan Lee, Suha Kwak, and Jeany Son. When to Switch: Reliable Action-Chunk Extension for Vision-Language-Action Models, 2026. arXiv:2610.05719.

[14] Yi Yu and Xinchuan Qiu. Benchmarking Vision-Language-Action Models on SO-101: Failure and Recovery Analysis, 2026. arXiv:2606.08881.

[15] Brian Zhu, Momen Khalil, E Harrison, Emanuele Poggi, Philipp Schmitt, Bernd Kast, Philine Meister, Pranav Atreya, Qiyang Li, Finn Ferchau, Cesar Colmenero, Yash Shahapurkar, Gokul Narayanan, Melih Erdogan, Kai Wurm, Georg von Wichert, Oier Mees, Eugen Solowjow, Andrew Wagenmaker, and Sergey Levine. Learning to Act While Waiting: RL Finetuning of Generalist Robot Policies Under Inference Latency, 2026. arXiv:2608.23831.

[16] Yixiang Zhu, Ruilong Wu, Yonghao Chen, Jiaxiang Zou, Yusong Hu, and Xinyu Chen. PACT-VLA: Speculative Action-Chunk Verification with Pre-Commit Observations, 2026. arXiv:2610.28019.

Appendix: from plain name to gate

Every number in this paper is recomputable from the repository; this table maps each experiment to its files.

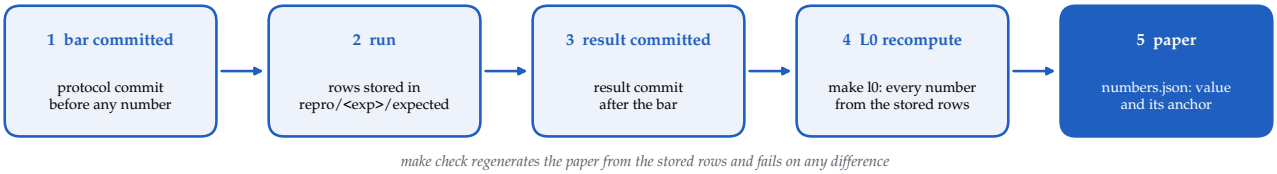


Figure 4: **How every number in this paper is made.** A protocol with its bar is committed before any number exists (1). The run stores its rows under `repro/` (2). The result is committed after the bar, and the commit order is audited (3). `make l0` recomputes every number from the stored rows on a CPU (4). `make_paper.py` writes `numbers.json`, with each value's anchor, and fills the text from it (5). `make check` regenerates the paper and fails on any difference.

trick	acts on	offline effect	bar	verdict	cost on the Orin	mode
prefix overwrite	seam	jump −71 / −73 / −72 % (vla.cpp, d = 8/12/20); PyTorch −73 / −74 / −72 %	cut ≥ 50 %	pass	+0.4 ms per chunk	25 W
exact RTC guidance	seam	jump −29 / −31 / −21 %	cut ≥ 50 %	fail	1962 vs 1028 ms naive (1.9×, PyTorch)	25 W
MP projection	within-chunk jerk	error -0.020°, jerk −94 %	error ≤ +0.30°	pass	< 1 ms	—
seam conditioning	seam jerk	−83 % (overlap 25) / −87 % (sync) at +0.19°	seam jerk ≤ 0.40×	pass	< 1 ms	—
two Euler steps	denoise loop	cloud ratio 0.78 (CI 0.73–0.81)	≤ 0.70	not proven	196.1 vs 237.5 ms per chunk	MAXN
speculative selection (PACT)	chunk choice	-0.27° vs a random pick (CI -0.44 to -0.09)	CI < 0, no added delay	selection pass, delay fail	1,200 ms per cycle, 37 ticks vs 8	MAXN
adaptive length (AAC)	chunk length	h* 2 / 2 (transport / grasp); jump 0.973 vs 0.940	transport ≥ grasp + 2; N = 20 ≤ 298 ms	fail	4,751 ms per decision	MAXN

Table 1: Seven tricks on one checkpoint: what each acts on, its offline effect, the pre-registered bar, the verdict and its cost on the Orin NX. MAXN = MAXN_SUPER.

experiment	repro folder	protocol commit		bar
serving split (25 W)	glat2	`82fdc15`		overhead ≤ 100 ms, ≤ 2 % over 238 ms
latency split, two Euler steps	horizon-depth	`0a79846`		cloud ratio ≤ 0.70; N2 ≤ 0.85 × N10
chunk boundary (overwrite, exact)	chunk-boundary	pre-registered (README)	2026-10-19	cut ≥ 50 %, drift ≤ +10 %, ≤ 667 ms
MP projection, conditioning	smooth-chunking	`6f58d7b`		error ≤ +0.30°; seam jerk ≤ 0.40×
speculative selection	pact	`0a79846`		ticks ≤ naive; beats random (CI < 0)
batched sampling control	pact	`1d7e6c4`		4 candidates cost ≥ 2.0 chunks
adaptive length	aac	`581a955`		transport h* ≥ grasp + 2; N20 ≤ 298 ms

Table 2: Plain name → `repro/` folder (its `expected/` files hold the rows) → the first commit of the gate's protocol → its bar.